

El Teorema de la Ecuación SAT y su Generalización

Dos manuscritos de Oscar Riveros en español

Oscar Riveros

Santiago, RM, Chile

oscar.riveros@gmail.com

Traducción al español de los dos manuscritos originales del autor: *A Unique Encoding of Satisfying Assignments for Balanced CNFs* (2025) y *A Generalization of the SAT Equation Theorem to Arbitrary CNF Formulas via Bitwise OR Encoding* (2025).

Primera parte

Una codificación única de las asignaciones satisfactores para CNF balanceadas

Oscar Riveros — Santiago, RM, Chile — oscar.riveros@gmail.com

Resumen

En este artículo introducimos y demostramos el Teorema de la Ecuación SAT, que establece que toda fórmula CNF balanceada sobre n variables booleanas queda caracterizada unívocamente por el número

$$S = \sum_{C \in \mathcal{F}} 2^{(\sum_{x \in C, x > 0} 2^{n-|x|})},$$

donde $\mathcal{F}$ es una fórmula CNF cuyas cláusulas son balanceadas (es decir, contienen exactamente un literal por variable) y la suma interna codifica el patrón de signos de C . Según nuestra convención de codificación, un bit con valor 1 en la expansión binaria de S indica una asignación que no satisface $\mathcal{F}$ (no satisfactoria), mientras que un 0 indica satisfacción. El teorema afirma que, escrita con al menos 2^n bits (de modo que todo vector booleano corresponda a una posición única), S codifica exactamente la tabla de verdad de $\mathcal{F}$. Presentamos aquí el enunciado detallado y su demostración, junto con observaciones sobre su importancia.

Conceptos CCS

- Teoría de la computación $\rightarrow$ Lógica.

Palabras clave

Satisfacibilidad; teoría de números.

1. Introducción

En la lógica proposicional, las fórmulas escritas en Forma Normal Conjuntiva (CNF) desempeñan un papel esencial. Una clase particular de fórmulas CNF, denominadas fórmulas balanceadas, es aquella en la que cada cláusula contiene exactamente un literal por variable. En una cláusula balanceada los n literales se eligen como $\pm 1, \pm 2, \dots, \pm n$ (sin repetición), de modo que cada variable aparece una vez, con signo positivo o negativo.

En este trabajo mostramos que para cualquier fórmula CNF balanceada $\mathcal{F}$ sobre n variables se puede definir el número

$$S = \sum_{C \in \mathcal{F}} 2^{(\sum_{x \in C, x > 0} 2^{n-|x|})},$$

que posee la propiedad notable de que su expansión binaria (escrita con al menos 2^n bits) codifica exactamente la tabla de verdad de $\mathcal{F}$, marcando con un 1 aquellas asignaciones que no satisfacen $\mathcal{F}$ y dejando ceros en todas las demás posiciones. En otras palabras, S determina unívocamente todas las asignaciones satisfactorias.

Este artículo presenta un enunciado claro y una demostración rigurosa del Teorema de Rive-ros, junto con una discusión de su importancia y sus posibles aplicaciones.

2. Preliminares

Suponemos que el lector está familiarizado con los conceptos básicos de la lógica proposicional. Una variable booleana x_i (para $i = 1, \dots, n$) toma valores en $\{0, 1\}$, donde adoptamos la convención de que 0 representa Verdadero y 1 representa Falso al evaluar fórmulas.

Un literal es una aparición positiva ($+i$) o una aparición negativa ($-i$) de la variable x_i . Una cláusula es una disyunción (OR) de literales; una fórmula CNF $\mathcal{F}$ es una conjunción (AND) de tales cláusulas. En una CNF balanceada, cada cláusula contiene exactamente un literal por variable. En consecuencia, cada cláusula $C \in \mathcal{F}$ puede representarse de manera única mediante su vector de signos:

$$v(C) = (v_1, v_2, \dots, v_n),$$

donde

$$v_i = \begin{cases} 1 & \text{si } +i \in C, \\ 0 & \text{si } -i \in C. \end{cases}$$

Se sigue inmediatamente que el número

$$T(C) = \sum_{i=1}^n v_i 2^{n-i}$$

es simplemente la representación binaria de $v(C)$ (con el bit más significativo correspondiente a la variable 1).

Además, bajo las reglas estándar de evaluación de las disyunciones, una cláusula es falsa si y solo si todos sus literales son falsos. Para las cláusulas balanceadas esto ocurre en exactamente un caso: si asignamos

$$a_C(i) = \begin{cases} 1 & \text{si } v_i = 1, \\ 0 & \text{si } v_i = 0, \end{cases}$$

entonces todo literal de C se evalúa como falso y, por tanto, C queda falsificada. Nótese que el número binario correspondiente a esta única asignación falsificante es exactamente $T(C)$.

3. Enunciado del teorema

Teorema 3.1. Sea $\mathcal{F}$ una fórmula CNF sobre n variables booleanas en la que toda cláusula es balanceada (es decir, cada cláusula contiene exactamente un literal por variable). Definimos

$$S = \sum_{C \in \mathcal{F}} 2^{(\sum_{x \in C, x > 0} 2^{n-|x|})}.$$

Si escribimos S en binario con al menos 2^n dígitos (de modo que cada asignación en $\{0,1\}^n$ corresponda a una posición de bit única), entonces el número resultante codifica la tabla de verdad de $\mathcal{F}$ respecto de las asignaciones satisfactorias: un 1 indica una asignación que no satisface $\mathcal{F}$ y un 0 indica satisfacción. En particular, S determina unívocamente todas las asignaciones satisfactorias.

4. Demostración

La demostración procede en tres pasos principales.

Paso 1: codificación de cada cláusula. Puesto que cada cláusula $C \in \mathcal{F}$ es balanceada, existe un único vector de signos

$$v(C) = (v_1, v_2, \dots, v_n), \quad v_i = \begin{cases} 1 & \text{si } +i \in C, \\ 0 & \text{si } -i \in C, \end{cases}$$

y por tanto

$$T(C) = \sum_{i=1}^n v_i 2^{n-i}.$$

Este $T(C)$ es simplemente la codificación binaria del patrón de signos de la cláusula C .

Paso 2: la única asignación falsificante. Bajo las reglas estándar de evaluación, una cláusula (por ser una disyunción) es falsa si y solo si todos sus literales son falsos. Para las cláusulas balanceadas esto ocurre de manera única cuando cada variable se asigna como sigue:

$$a_C(i) = \begin{cases} 1 & \text{si } v_i = 1 \quad (\text{pues } +i \text{ se vuelve falsa}), \\ 0 & \text{si } v_i = 0 \quad (\text{pues } -i \text{ se vuelve falsa}). \end{cases}$$

Así, la única asignación a_C que falsifica la cláusula C corresponde exactamente al número binario $T(C)$.

Paso 3: formación e interpretación de S . Por definición,

$$S = \sum_{C \in \mathcal{F}} 2^{T(C)}.$$

Supongamos que $\mathcal{F}$ no contiene cláusulas repetidas (si hubiera duplicados, puede adaptarse el argumento para tomar en cuenta los acarreos de la suma binaria). Entonces cada cláusula contribuye con una potencia de dos distinta. Al escribir S en base 2 con al menos 2^n dígitos, las únicas posiciones en las que aparece un 1 son las que corresponden a algún $T(C)$. Es decir, una asignación $a \in \{0,1\}^n$ aparece como un 1 (esto es, no satisfactoria) si y solo si existe una cláusula C con $T(C) = a$.

Puesto que una asignación satisface $\mathcal{F}$ precisamente cuando ninguna de sus cláusulas queda falsificada (es decir, cuando el bit correspondiente en la expansión binaria es 0), se sigue que la representación binaria de S codifica exactamente la tabla de verdad de $\mathcal{F}$, con unos marcando las asignaciones no satisfactorias y ceros las satisfactorias. Así, a partir de S se pueden recuperar unívocamente todas las asignaciones satisfactorias.

5. Conclusión

Hemos mostrado que para cualquier fórmula CNF balanceada sobre n variables booleanas el número

$$S = \sum_{C \in \mathcal{F}} 2^{(\sum_{x \in C, x > 0} 2^{n-|x|})}$$

codifica la tabla de verdad de $\mathcal{F}$ de manera única. Según nuestra convención de codificación, cada bit corresponde a una asignación (con 1 significando no satisfecha y 0 satisfecha), de modo que S caracteriza por completo el conjunto de asignaciones satisfactorias. Este resultado no solo ofrece una representación compacta para las fórmulas CNF balanceadas, sino que también aporta nuevas perspectivas sobre su estructura, con aplicaciones potenciales en el razonamiento automatizado y en la síntesis lógica.

Segunda parte

Generalización del Teorema de la Ecuación SAT a fórmulas CNF arbitrarias mediante codificación con OR bit a bit

Oscar Riveros — Santiago, RM, Chile — oscar.riveros@gmail.com

Resumen

El Teorema de la Ecuación SAT establece que toda fórmula CNF balanceada sobre n variables booleanas queda caracterizada unívocamente por el entero

$$S = \sum_{C \in \mathcal{F}} 2^{(\sum_{x \in C, x > 0} 2^{n-|x|})},$$

cuya expansión binaria codifica la tabla de verdad de la fórmula: un 1 marca una asignación no satisfaciente y un 0 una satisfaciente. Sin embargo, la formulación original está restringida a las cláusulas balanceadas, aquellas que contienen exactamente un literal por variable, porque la suma aritmética introduce acarreo cuando varias cláusulas comparten la misma asignación falsificante.

En este artículo generalizamos el teorema a fórmulas CNF arbitrarias (con cláusulas de cualquier tamaño) reemplazando la suma aritmética por la operación OR bit a bit. Bajo esta codificación, la contribución de cada cláusula es una máscara binaria que pone un 1 en cada posición correspondiente a una asignación que falsifica esa cláusula. El OR de todas las máscaras produce un número S cuya expansión binaria registra, sin acarreo, exactamente la tabla de verdad de la fórmula completa. Proporcionamos un enunciado completo, su demostración y ejemplos ilustrativos de esta generalización.

Conceptos CCS

- Teoría de la computación $\rightarrow$ Lógica.

Palabras clave

Satisfacibilidad, SAT, CNF, funciones booleanas, teoría de números, codificación por tabla de verdad.

1. Introducción

En la lógica proposicional, las fórmulas escritas en Forma Normal Conjuntiva (CNF) son ubicuas en el razonamiento automatizado, la verificación formal y la teoría de la complejidad. Una fórmula CNF $\mathcal{F}$ sobre n variables booleanas $x_1, x_2, \dots, x_n$ es una conjunción de cláusulas, cada una de las cuales es una disyunción de literales. Un literal es una aparición positiva $+i$ o una aparición negativa $-i$ de la variable x_i .

Una fórmula CNF balanceada es aquella en la que cada cláusula contiene exactamente un literal por variable; es decir, cada cláusula es una disyunción completa de n literales, uno sobre cada variable distinta. Para tales fórmulas, Riveros [1] demostró el siguiente resultado notable:

Teorema 1.1 (Teorema de la Ecuación SAT, caso balanceado). *Sea $\mathcal{F}$ una fórmula CNF balanceada sobre n variables booleanas. Definimos*

$$S = \sum_{C \in \mathcal{F}} 2^{(\sum_{x \in C, x > 0} 2^{n-|x|})}.$$

Si S se escribe en binario con al menos 2^n dígitos, entonces el k -ésimo bit es 1 si y solo si la asignación codificada por la representación binaria de k no satisface $\mathcal{F}$. En consecuencia, S codifica unívocamente la tabla de verdad de $\mathcal{F}$.

La codificación es elegante y puramente aritmética; sin embargo, el enunciado original del teorema se limita a las cláusulas balanceadas. Al intentar extender la construcción a cláusulas que omiten algunas variables, surgen dos dificultades:

1. Una cláusula a la que le faltan t variables no es falsificada por una única asignación, sino por un subcubo completo de 2^t asignaciones. Por tanto, debe codificarse un *conjunto* de posiciones y no una posición única.
2. Aun si cada cláusula se codificara como la suma de potencias de dos sobre su subcubo falsificante, la sumatoria aritmética sobre las distintas cláusulas puede producir acarreo ($2^a + 2^a = 2^{a+1}$), rompiendo la correspondencia biunívoca entre bits y asignaciones.

En este artículo resolvemos ambas dificultades reemplazando la suma aritmética por la operación OR bit a bit. Esto produce una generalización limpia y libre de acarreo que se aplica a cualquier fórmula CNF, independientemente del tamaño o de la estructura de sus cláusulas.

2. Preliminares

Trabajamos con N variables booleanas $x_1, x_2, \dots, x_N$, cada una de las cuales toma valores en $\{0, 1\}$. Adoptamos la convención de que 0 representa Verdadero y 1 representa Falso al evaluar literales. Un literal es $+i$ (positivo) o $-i$ (negativo). Una cláusula C es una disyunción de literales; una fórmula CNF $\mathcal{F} = C_1 \wedge C_2 \wedge \dots \wedge C_m$ es una conjunción de cláusulas.

Definición 2.1 (Posición de una asignación). Para una asignación $a = (a_1, a_2, \dots, a_N) \in \{0, 1\}^N$, su posición es el entero

$$\text{pos}(a) = \sum_{i=1}^N a_i \cdot 2^{N-i}.$$

Así, pos es una biyección entre $\{0, 1\}^N$ y $\{0, 1, \dots, 2^N - 1\}$, interpretada como los índices de los bits de un número binario de 2^N bits.

Definición 2.2 (Variables presentes y ausentes en una cláusula). Para una cláusula C , sean

$$P_C = \{i \in \{1, \dots, N\} \mid +i \in C \text{ o } -i \in C\}$$

(variables presentes en C) y

$$M_C = \{1, \dots, N\} \setminus P_C$$

(variables ausentes de C).

Definición 2.3 (Bit requerido). Para cada variable presente $i \in P_C$, el bit requerido $r_i(C)$ es el valor que vuelve falso el literal:

$$r_i(C) = \begin{cases} 0, & \text{si } +i \in C \quad (\text{pues } +i \text{ es falsa cuando } x_i = 0), \\ 1, & \text{si } -i \in C \quad (\text{pues } -i \text{ es falsa cuando } x_i = 1). \end{cases}$$

Con estas definiciones, una cláusula C es falsa exactamente cuando $x_i = r_i(C)$ para todo $i \in P_C$. Las variables ausentes M_C pueden tomar cualquier valor sin afectar la falsedad de C .

Definición 2.4 (Contribución base de una cláusula). La contribución base $B(C)$ codifica los bits requeridos de las variables presentes:

$$B(C) = \sum_{i \in P_C} r_i(C) \cdot 2^{N-i}.$$

3. Codificación de una cláusula mediante OR

Sea C una cláusula y sea $M_C = \{i_1, i_2, \dots, i_t\}$ el conjunto de sus variables ausentes, donde $t = |M_C| = N - |P_C|$. El conjunto de todas las asignaciones que falsifican a C es precisamente

$$\text{Fals}(C) = \{a \in \{0, 1\}^N \mid a_i = r_i(C) \ \forall i \in P_C\}.$$

Para cada subconjunto $M' \subseteq M_C$ obtenemos una asignación falsificante distinta fijando las variables de M' en 1 y las variables ausentes restantes en 0. La posición de esta asignación es

$$B(C) + \sum_{i \in M'} 2^{N-i}.$$

Por tanto, la contribución de la cláusula C a la codificación global es el OR bit a bit de 2^t potencias de dos distintas:

$$S_C = \bigvee_{M' \subseteq M_C} 2^{B(C) + \sum_{i \in M'} 2^{N-i}},$$

donde $\bigvee$ denota la operación OR bit a bit. Equivalentemente, S_C es el número binario cuyo k -ésimo bit es 1 si y solo si $k = B(C) + \sum_{i \in M'} 2^{N-i}$ para algún $M' \subseteq M_C$.

Observación 3.1. En el caso balanceado, $P_C = \{1, \dots, N\}$ y $M_C = \emptyset$. Entonces $S_C = 2^{B(C)} = 2^{T(C)}$, lo que recupera la codificación original.

4. Codificación de una fórmula CNF

Una fórmula CNF $\mathcal{F} = C_1 \wedge C_2 \wedge \dots \wedge C_m$ es falsificada por una asignación a si y solo si al menos una cláusula C_j es falsificada por a . En consecuencia, el conjunto de todas las asignaciones falsificantes es la unión de los conjuntos falsificantes de las cláusulas individuales.

Puesto que cada S_{C_j} tiene un 1 precisamente en las posiciones correspondientes a $\text{Fals}(C_j)$, el número que codifica la unión es simplemente el OR bit a bit de las contribuciones individuales:

$$S = \bigvee_{j=1}^m S_{C_j}.$$

Combinando los dos niveles, obtenemos la forma totalmente expandida:

$$S = \bigvee_{j=1}^m \bigvee_{M' \subseteq M_{C_j}} 2^{B(C_j) + \sum_{i \in M'} 2^{N-i}}.$$

Esta es la definición central de este artículo.

5. El Teorema de la Ecuación SAT generalizado

Teorema 5.1 (Teorema de la Ecuación SAT generalizado). Sea $\mathcal{F}$ una fórmula CNF arbitraria sobre N variables booleanas, con m cláusulas de cualquier tamaño. Sea S definido como arriba mediante la codificación OR bit a bit. Entonces, cuando S se escribe en binario con 2^N bits, se cumple lo siguiente para toda asignación $a \in \{0, 1\}^N$: el bit en la posición $\text{pos}(a)$ es

$$\begin{cases} 1, & \text{si } a \text{ no satisface } \mathcal{F}, \\ 0, & \text{si } a \text{ satisface } \mathcal{F}. \end{cases}$$

En particular, S es una codificación entera única y exacta de la tabla de verdad de $\mathcal{F}$.

Demostración. La demostración se sigue directamente de la construcción.

Codificación de una cláusula. Por definición, para cada cláusula C_j ,

$$S_{C_j} = \bigvee_{M' \subseteq M_{C_j}} 2^{B(C_j) + \sum_{i \in M'} 2^{N-i}}.$$

Todos los exponentes son distintos porque cada subconjunto M' produce una suma de pesos diferente. Por tanto, S_{C_j} es un número binario bien definido en el que el k -ésimo bit es 1 si y solo si $k = \text{pos}(a)$ para alguna $a \in \text{Fals}(C_j)$. Así, S_{C_j} codifica exactamente el conjunto de asignaciones que falsifican a C_j .

Codificación de la fórmula completa. Una asignación a satisface $\mathcal{F}$ si y solo si satisface todas las cláusulas C_j , es decir, si y solo si $a \notin \bigcup_{j=1}^m \text{Fals}(C_j)$. Equivalentemente, a falsifica $\mathcal{F}$ si y solo si existe j tal que $a \in \text{Fals}(C_j)$.

Ahora bien,

$$S = \bigvee_{j=1}^m S_{C_j}.$$

Puesto que el OR corresponde a la unión de conjuntos sobre las posiciones de bit, el k -ésimo bit de S es 1 si y solo si existe algún j para el cual el k -ésimo bit de S_{C_j} es 1, es decir, si y solo si existe alguna cláusula C_j y alguna asignación $a \in \text{Fals}(C_j)$ con $\text{pos}(a) = k$. Esta es precisamente la condición de que a falsifique a $\mathcal{F}$.

Ausencia de acarreo. Como la operación OR es idempotente ($1 \vee 1 = 1$), si dos cláusulas distintas (o dos subconjuntos M' de la misma cláusula) fijan el mismo bit, el resultado sigue siendo 1. No se generan acarreo. La correspondencia biunívoca entre posiciones de bit y asignaciones queda así preservada incondicionalmente.

Conclusión. La expansión binaria de S tiene un 1 en la posición $\text{pos}(a)$ si y solo si a es una asignación no satisfaciente, y 0 en caso contrario. Por tanto, S codifica unívocamente la tabla de verdad de $\mathcal{F}$. $\square$

6. Ejemplos ilustrativos

6.1. Caso balanceado (recuperando el teorema original)

Sea $N = 3$ y

$$\mathcal{F} = (x_1 \vee x_2 \vee \neg x_3) \wedge (\neg x_1 \vee x_2 \vee x_3).$$

- C_1 : $+x_1, +x_2, -x_3$. Entonces $P_{C_1} = \{1, 2, 3\}$ y $M_{C_1} = \emptyset$. Los bits requeridos son $r_1 = 0$, $r_2 = 0$, $r_3 = 1$, de donde $B(C_1) = 0 \cdot 2^2 + 0 \cdot 2^1 + 1 \cdot 2^0 = 1$. Por tanto, $S_{C_1} = 2^1 = 2$ (binario 00000010; el bit 1 es 1).
- C_2 : $-x_1, +x_2, +x_3$. Los bits requeridos son $r_1 = 1$, $r_2 = 0$, $r_3 = 0$, de donde $B(C_2) = 1 \cdot 2^2 + 0 \cdot 2^1 + 0 \cdot 2^0 = 4$. Por tanto, $S_{C_2} = 2^4 = 16$ (binario 00010000; el bit 4 es 1).

Tenemos $S = 2 \vee 16 = 18$ (binario 00010010). Los bits 1 y 4 son 1, lo que significa que las asignaciones $(0, 0, 1)$ y $(1, 0, 0)$ falsifican a $\mathcal{F}$; las otras 6 asignaciones satisfacen a $\mathcal{F}$. Esto coincide con el teorema original.

6.2. Cláusula no balanceada

Sea $N = 3$ y supongamos que $\mathcal{F}$ consta de una única cláusula $C = (x_1 \vee \neg x_2)$, con x_3 ausente.

- $P_C = \{1, 2\}$ y $M_C = \{3\}$.

- $r_1 = 0$ (por $+x_1$) y $r_2 = 1$ (por $-x_2$).
- $B(C) = 0 \cdot 2^2 + 1 \cdot 2^1 = 2$.
- Peso de x_3 : $2^{3-3} = 2^0 = 1$.

Subconjuntos de $M_C = \{3\}$:

$$M' = \emptyset : 2^{B(C)+0} = 2^2 = 4,$$

$$M' = \{3\} : 2^{B(C)+1} = 2^3 = 8.$$

Por tanto, $S = 4 \vee 8 = 12$ (binario 00001100).

Los bits en las posiciones 2 y 3 son 1:

- Posición 2 = $(0, 1, 0)$: $x_1 = 0$, $x_2 = 1$. En C , $+x_1$ es falsa y $-x_2$ es falsa, de modo que la cláusula es falsa. ✓
- Posición 3 = $(0, 1, 1)$: lo mismo para x_1 y x_2 , con x_3 arbitraria. La cláusula es falsa. ✓

Todas las demás asignaciones satisfacen a C . Este ejemplo ilustra cómo una única variable ausente duplica el número de posiciones falsificantes (un subcubo unidimensional).

6.3. Dos cláusulas que comparten una asignación falsificante

Sea $N = 3$ y

$$\mathcal{F} = (x_1 \vee \neg x_2) \wedge (\neg x_1 \vee x_2 \vee x_3).$$

- C_1 : $P_{C_1} = \{1, 2\}$, $M_{C_1} = \{3\}$ y $B(C_1) = 0 \cdot 4 + 1 \cdot 2 = 2$. Entonces $S_{C_1} = 2^2 \vee 2^{2+1} = 4 \vee 8 = 12$.
- C_2 : $P_{C_2} = \{1, 2, 3\}$, $M_{C_2} = \emptyset$, $r_1 = 1$, $r_2 = 0$, $r_3 = 0$ y $B(C_2) = 4$. Entonces $S_{C_2} = 2^4 = 16$.

Tenemos $S = 12 \vee 16 = 28$ (binario 00011100).

Los bits 2, 3 y 4 son 1. La asignación $(1, 0, 0)$ (posición 4) falsifica solo a C_2 ; las asignaciones $(0, 1, 0)$ y $(0, 1, 1)$ (posiciones 2 y 3) falsifican a C_1 . No hay solapamiento entre las contribuciones de las cláusulas, pero aunque lo hubiera, la operación OR lo manejaría sin acarreo.

7. Discusión

7.1. Por qué el OR es esencial

La ventaja crucial del OR bit a bit sobre la suma aritmética es la idempotencia: $1 \vee 1 = 1$, mientras que $2^a + 2^a = 2^{a+1}$ produce un acarreo que desplaza el bit a una posición superior, destruyendo la correspondencia entre los índices de bit y las asignaciones. Con el OR, las asignaciones falsificantes solapadas de distintas cláusulas se manejan de manera implícita y correcta.

7.2. Alternativa aritmética con base $b > m$

Si por razones teóricas se insistiera en usar la suma aritmética, los acarreo pueden evitarse eligiendo una base b estrictamente mayor que el número máximo de cláusulas que pueden compartir una posición falsificante dada. Como cada asignación puede ser falsificada por a lo sumo m cláusulas, usando

$$S = \sum_{j=1}^m \sum_{M' \subseteq M_{C_j}} b^{B(C_j) + \sum_{i \in M'} 2^{N-i}}$$

con $b > m$, se garantiza que ningún dígito supere $b - 1$ y no se producen acarreo. Sin embargo, esto produce una representación en base b en lugar de una representación binaria, y es menos natural que la formulación con OR.

7.3. Recuperación de las asignaciones satisficentes a partir de S

Dado S , se puede comprobar si una asignación a satisface a $\mathcal{F}$ simplemente inspeccionando el bit en la posición $\text{pos}(a)$ de S :

$$a \text{ satisface a } \mathcal{F} \iff \left(\left\lfloor S/2^{\text{pos}(a)} \right\rfloor \bmod 2 \right) = 0.$$

Para enumerar todas las asignaciones satisficentes, se itera sobre $k = 0, 1, \dots, 2^N - 1$ y se recogen aquellos k para los cuales el k -ésimo bit de S es 0. Este es un procedimiento directo.

7.4. Consideraciones computacionales

El entero S puede ser tan grande como $2^{2^N} - 1$, lo que requiere 2^N bits para representarlo. Si bien esto es exponencial en N , el teorema es un resultado estructural sobre la función booleana codificada por una fórmula CNF, y no una afirmación algorítmica sobre su manipulación eficiente. Proporciona una descripción combinatoria compacta de la tabla de verdad y puede encontrar aplicaciones en áreas como:

- Pruebas de unicidad para codificaciones SAT.
- Caracterizaciones algebraicas de funciones booleanas.
- Conexiones teóricas entre la teoría de números y la lógica proposicional.

8. Conclusión

Hemos generalizado el Teorema de la Ecuación SAT de las fórmulas CNF balanceadas a fórmulas CNF arbitrarias reemplazando la suma aritmética por la operación OR bit a bit. La construcción resultante produce un único entero S , definido por

$$S = \bigvee_{j=1}^m \bigvee_{M' \subseteq M_{C_j}} 2^{B(C_j) + \sum_{i \in M'} 2^{N-i}},$$

cuya expansión binaria de 2^N bits codifica exactamente la tabla de verdad de la fórmula: los bits en las posiciones correspondientes a asignaciones no satisficentes son 1, y todos los demás bits son 0. Esta codificación es única, libre de acarreo y se aplica sin restricciones a cualquier fórmula CNF sobre cualquier número de variables.

La técnica del OR bit a bit resuelve la dificultad fundamental que plantean las cláusulas no balanceadas —a saber, que una cláusula a la que le faltan t variables genera un subcubo t -dimensional de asignaciones falsificantes en lugar de una sola— y al mismo tiempo elimina el problema de los acarreo inherente a la sumatoria aritmética. Creemos que esta generalización abre nuevas vías para explorar conexiones entre la satisfacibilidad proposicional y las representaciones numéricas de las funciones booleanas.

Referencias

- [1] Oscar Riveros. *A Unique Encoding of Satisfying Assignments for Balanced CNFs*. Publicación de investigación en Internet, 2025.